

Catfish — Clue Graph Schema (v0.1)

Core insight

A mystery where the killer is randomized can't just randomize *who* did it. It has to randomize *what the evidence means*. That means three layers of content, not one:

Layer	Lives where	Changes per run?
Static facts	Ground-truth content. Every run has them.	No
Variable facts	Content whose payload depends on the killer.	Yes — payload swapped at run init
Conditional facts	Facts that only exist if a specific character is the killer.	Yes — presence toggled

The player sees everything as a uniform stream. The author sees three clearly-separated buckets.

Layer 0: Identity

```
enum CharacterID: String, Codable, CaseIterable, Identifiable {  
    case kai, river, miles, sam, jules  
    var id: String { rawValue }  
}
```

```
enum FactSource: Codable, Hashable {  
    case directMessage(from: CharacterID)  
    case socialPost(author: CharacterID)  
    case friendText(from: FriendID)  
    case dateScene(with: CharacterID)  
    case profileBio(of: CharacterID)  
}
```

```
enum FriendID: String, Codable { case dev, morgan }
```

Layer 1: The Run (World Seed)

One `CaseRun` per playthrough. This is your bug-report anchor — every save, crash log, and screenshot should be able to reference `runID` and replay.

```
@Model
final class CaseRun {
    @Attribute(.unique) var runID: UUID
    var rngSeed: UInt64 // for non-killer randomization
    var killerIDRaw: String // CharacterID.rawValue, stored as S
    var startedAt: Date
    var currentDay: Int // 1...7
    var ended: CaseEnding? // nil until resolved

    @Relationship(deleteRule: .cascade, inverse: \DiscoveredFact.run)
    var discovered: [DiscoveredFact] = []

    init(killer: CharacterID, seed: UInt64 = .random(in: .min ... .max)) {
        self.runID = UUID()
        self.rngSeed = seed
        self.killerIDRaw = killer.rawValue
        self.startedAt = .now
        self.currentDay = 1
    }

    var killerID: CharacterID { CharacterID(rawValue: killerIDRaw)! }
}

enum CaseEnding: String, Codable {
    case caughtThem // correct accusation
    case wrongfulAccusation // named an innocent
    case metKiller // agreed to the Day 7 meet
    case killerEscaped // refused to meet, didn't accuse
}
```

Layer 2: Facts

A `Fact` is a discoverable unit of content. The player sees its `payload`; they don't see whether it's static, variable, or conditional.

```
enum FactKind: String, Codable {
    case staticFact // same in every run
    case variable // payload swapped based on killer
}
```

```

        case conditional // only exists when specific char is killer
    }

    struct FactPayload: Codable, Hashable {
        var text: String?
        var imageAssetID: String? // maps to catfish_asset_prompts.csv asset
        var voiceLineID: String? // pre-generated ElevenLabs key
        var metadata: [String: String] = [:] // e.g. ["timestamp": "Day 3, 9:14pm"]
    }

    @Model
    final class Fact {
        @Attribute(.unique) var id: String // stable authoring ID, e.g. "kai_da
        var kindRaw: String // FactKind
        var sourceData: Data // encoded FactSource
        var day: Int // when it becomes available
        var aboutCharacterRaw: String? // CharacterID this fact concerns
        var payloadData: Data // encoded FactPayload (current reso

        var kind: FactKind { FactKind(rawValue: kindRaw)! }
        var source: FactSource {
            (try? JSONDecoder().decode(FactSource.self, from: sourceData)) ?? .profil
        }
        var payload: FactPayload {
            (try? JSONDecoder().decode(FactPayload.self, from: payloadData)) ?? FactP
        }
    }

    @Model
    final class DiscoveredFact {
        var run: CaseRun?
        var factID: String
        var discoveredAt: Date
        var playerNote: String?
        var linkedFactIDs: [String] = [] // journal cross-references
    }

```

Layer 3: KillerIdentity protocol — the guilt rules

Each of the five characters has a `KillerIdentity` implementation. At run init, exactly one is instantiated and stamped onto the `CaseRun`. Everything downstream (which variable facts get which payload, which conditional facts get inserted, which friend texts fire) resolves from this one object.

```

protocol KillerIdentity {
    var characterID: CharacterID { get }

    /// Fact IDs that are ONLY present when this character is the killer.
    /// Typically: the "uneasy" portrait, the contradicting social post,
    /// friend-text warnings pointing here.
    var conditionalFactIDs: Set<String> { get }

    /// Fact IDs whose payload is overridden by this killer.
    /// The Fact exists in every run; this map provides THIS killer's version.
    var variableOverrides: [String: FactPayload] { get }

    /// The canonical "solution" deduction – the minimal fact set that proves gui
    /// Used at accusation time to score whether the player has the evidence to b
    var solvingDeduction: Deduction { get }

    /// Red-herring deduction chains this killer plants to mislead the player
    /// toward accusing someone else. Used to wire the "wrongful accusation" endi
    var redHerrings: [Deduction] { get }
}

struct Deduction: Codable, Hashable {
    let id: String
    let requiredFactIDs: Set<String> // must all be discovered
    let impliedKiller: CharacterID // who this chain accuses
    let weight: Double // 0...1 how strong the chain is
    let narrativeBeat: String // shown to player post-accusation
}

```

The Double-Blind Tell, wired concretely

Scenario the red-team called out. Worked example with two of the five killers:

Universal static fact — always present:

```

id: "miles_bio_downtown_view"
source: .profileBio(of: .miles)
payload.text: "Finance guy, downtown loft, amazing skyline view."

```

Variable fact — "miles_ig_window_reflection". Same fact exists in every run. Its *payload* depends on the killer.

- When **Miles is killer**: payload shows an IG story with a reflection revealing he's in a

suburban apartment, not downtown. This contradicts the bio → the solving chain fires.

- When **Kai is killer**: payload shows a generic downtown view, consistent with bio. Ambient content. No tell.
- When **Jules is killer**: payload shows downtown view, BUT includes a visible landmark that matches the location where Jules claimed to be at the same time → red herring pointing at Miles.

Three implementations, three different payloads for the same `Fact.id`:

```
struct MilesIsKiller: KillerIdentity {
  let characterID: CharacterID = .miles

  var conditionalFactIDs: Set<String> = [
    "miles_portrait_uneasy_day5",
    "dev_text_day4_miles_sus",
    "miles_day6_location_ping_mismatch"
  ]

  var variableOverrides: [String: FactPayload] = [
    "miles_ig_window_reflection": FactPayload(
      text: "Late night grind 🏠",
      imageAssetID: "miles_post4_recent_milesKiller",
      metadata: ["tell": "reflection shows suburban parking lot, not downto
    )
  ]

  var solvingDeduction = Deduction(
    id: "miles_solution",
    requiredFactIDs: [
      "miles_bio_downtown_view",
      "miles_ig_window_reflection",
      "dev_text_day4_miles_sus"
    ],
    impliedKiller: .miles,
    weight: 1.0,
    narrativeBeat: "The reflection doesn't lie. He was never downtown."
  )

  var redHerrings: [Deduction] = [
    // Miles-as-killer plants a chain pointing at Jules
    Deduction(
      id: "miles_red_herring_jules",
      requiredFactIDs: ["jules_day2_late_reply", "jules_bio_night_owl"],
      impliedKiller: .jules,
      weight: 0.4,
```

```

        narrativeBeat: "You thought the night owl was stalking you. He was ju
    )
]
}

struct JulesIsKiller: KillerIdentity {
    let characterID: CharacterID = .jules

    var conditionalFactIDs: Set<String> = [
        "jules_portrait_uneasy_day4",
        "morgan_text_day5_jules_google",
        "jules_day7_meet_location_isolated"
    ]

    var variableOverrides: [String: FactPayload] = [
        "miles_ig_window_reflection": FactPayload(
            text: "View from the corner office ",
            imageAssetID: "miles_post4_recent_julesKiller",
            metadata: ["planted_tell": "visible landmark matches Jules's alibi lo
        )
    ]

    var solvingDeduction = Deduction(
        id: "jules_solution",
        requiredFactIDs: [
            "jules_bio_contradiction",
            "morgan_text_day5_jules_google",
            "jules_day6_chat_slip"
        ],
        impliedKiller: .jules,
        weight: 1.0,
        narrativeBeat: "The Google search Morgan found. No one searches that for
    )

    var redHerrings: [Deduction] = [
        // Jules-as-killer plants a chain pointing at Miles via that same photo
        Deduction(
            id: "jules_red_herring_miles",
            requiredFactIDs: ["miles_bio_downtown_view", "miles_ig_window_reflect
            impliedKiller: .miles,
            weight: 0.6,
            narrativeBeat: "The finance guy seemed shady. He was just vain."
        )
    ]
}

```

The same `Fact.id` resolves to different content, and the same fact set fires a different deduction, depending on which `KillerIdentity` is active. That's the double-blind.

Run initialization

```
enum KillerRegistry {
    static func identity(for id: CharacterID) -> KillerIdentity {
        switch id {
        case .kai:    return KaiIsKiller()
        case .river:  return RiverIsKiller()
        case .miles:  return MilesIsKiller()
        case .sam:    return SamIsKiller()
        case .jules:  return JulesIsKiller()
        }
    }
}

@MainActor
final class RunBootstrapper {
    let context: ModelContext

    init(context: ModelContext) { self.context = context }

    func startNewRun(killer: CharacterID? = nil) throws -> CaseRun {
        let chosen = killer ?? CharacterID.allCases.randomElement()!
        let run = CaseRun(killer: chosen)
        context.insert(run)

        let identity = KillerRegistry.identity(for: chosen)
        try applyIdentity(identity, to: run)
        try context.save()
        return run
    }

    private func applyIdentity(_ identity: KillerIdentity, to run: CaseRun) throw
        // 1. Load the universe of authored facts from bundled JSON
        let allFacts = try FactAuthoringLoader.loadAll()

        // 2. For each fact, decide inclusion + payload
        for authored in allFacts {
            switch authored.kind {
            case .staticFact:
                context.insert(Fact(from: authored))
            }
        }
    }
}
```

```

        case .variable:
            var fact = Fact(from: authored)
            if let override = identity.variableOverrides[authored.id] {
                fact.payloadData = try JSONEncoder().encode(override)
            }
            context.insert(fact)

        case .conditional:
            guard identity.conditionalFactIDs.contains(authored.id) else { co
            context.insert(Fact(from: authored))
            }
        }
    }
}

```

`FactAuthoringLoader` reads from bundled JSON so writers iterate without touching Swift. Killswitch-friendly if you want to push fact updates via your Cloudflare Worker later.

Accusation resolution

```

struct AccusationResult {
    let correct: Bool
    let matchedDeduction: Deduction?
    let ending: CaseEnding
    let narrativeBeat: String
}

func resolveAccusation(
    accused: CharacterID,
    run: CaseRun,
    discovered: Set<String>
) -> AccusationResult {
    let truth = KillerRegistry.identity(for: run.killerID)

    if accused == run.killerID {
        let hasEvidence = truth.solvingDeduction.requiredFactIDs.isSubset(of: discovered)
        return AccusationResult(
            correct: true,
            matchedDeduction: hasEvidence ? truth.solvingDeduction : nil,
            ending: .caughtThem,
            narrativeBeat: hasEvidence
                ? truth.solvingDeduction.narrativeBeat
                : "You were right. You just got lucky."
        )
    }
}

```



```

    }

    // Wrong accusation — check if a red herring "justifies" it narratively
    let matched = truth.redHerrings.first { rh in
        rh.implicitKiller == accused &&
        rh.requiredFactIDs.isSubset(of: discovered)
    }

    return AccusationResult(
        correct: false,
        matchedDeduction: matched,
        ending: .wrongfulAccusation,
        narrativeBeat: matched?.narrativeBeat
            ?? "You pointed at a stranger. The real one kept swiping."
    )
}

```

The “lucky guess” vs “earned it” distinction is worth preserving — a player who accuses without evidence gets a slightly deflated correct ending, which rewards the investigation without punishing the intuition.

Authoring workflow (writer-facing)

Each fact is one JSON object. The writer’s job: author the universe in JSON, annotate which are variable, provide per-killer overrides in the identity Swift files.

```

{
  "id": "miles_ig_window_reflection",
  "kind": "variable",
  "source": { "socialPost": { "author": "miles" } },
  "day": 3,
  "aboutCharacter": "miles",
  "payload": {
    "text": "End of a long week.",
    "imageAssetID": "miles_post4_recent_default",
    "metadata": {}
  }
}

```

Default payload lives in JSON. Killer-specific overrides live in the five `*IsKiller.swift` files, so diffing a killer file shows you exactly what that identity changes about the world. That separation is the whole maintainability story — writers can read one Swift file and know the full “here’s how this character being guilty manifests.”

Open questions for paper-prototype week

1. **Minimum solvable fact count per killer.** Gut says 3 facts in `solvingDeduction.requiredFactIDs`. Too few and it's trivial; too many and Day 7 comes too fast.
2. **Red herring count per killer.** One or two? More than two and the wrongful-accusation ending fires too easily.
3. **Does discovered include facts the player saw or only facts they *logged*?** Logging (drag to journal) is the commit action. Recommend: only logged facts count toward deductions. Forces engagement.
4. **Friend DDA trigger rule.** Probably "if player has logged $< N$ facts by end of Day X, friend texts a pointed hint." Needs tuning pass.

What this does NOT do yet

- Message threading / conversation state (separate model — chats are time-series, facts are flags)
- Inworld ambient dialogue wiring (the state-gated context handoff lives at the chat layer, not the clue layer)
- Voice-line bundling / preload queue (audio engine concern)
- Social feed ordering within a character's profile (presentation-layer concern)

These are the next three schemas. The clue graph is the spine; everything else hangs off it.